

Distributed Systems

Mid-1 Exam - Important Topics & Answers

Definitions, explanations, and examples for the six key topics.

- 1. Define Distributed System
- 2. Implementation of RPC
- 3. Difference between Client and Server
- 4. What is the Name Space
- 5. Explain Election Algorithm
- 6. Overview of Distributed Operating Systems

1. Define Distributed System

A **distributed system** is a collection of independent computers that appears to its users as a single coherent system. The computers (nodes) are physically separate, connected by a network, but coordinate their actions by passing messages, and work together to achieve a common goal without the user needing to know which machine actually did the work.

Key characteristics:

- **No shared memory** - nodes communicate only via message passing over a network
- **Concurrency** - multiple nodes execute simultaneously
- **No global clock** - each node has its own clock, so exact global ordering of events is hard
- **Independent failures** - one node can fail while others keep running

Example: Google Search runs across thousands of servers worldwide. When you search, you don't know (or care) which data center or machine answered - it just looks like "one search engine." Other examples: DNS, Amazon's e-commerce backend, Google Docs (multiple users editing the same document from different machines).

2. Implementation of RPC (Remote Procedure Call)

RPC lets a program call a procedure/function that runs on a different machine, while making it *look* like a normal local function call. The goal is to hide network communication details from the programmer.

How it's implemented, step by step:

- **Client calls a local "stub"** - the client program calls what looks like an ordinary local function (e.g., `getBalance(acctId)`).
- **Client stub (client proxy)** packages the function name and arguments into a message - this step is called **marshalling** (serialization).
- **Message is sent over the network** to the server, using an underlying transport protocol (TCP/UDP).
- **Server stub (skeleton)** receives the message and **unmarshals** it (deserializes the arguments).
- **Server stub calls the actual procedure** on the server with the unmarshalled arguments.
- **Server executes the procedure** and produces a result.
- **Server stub marshals the result** into a message and sends it back to the client.
- **Client stub unmarshals the result** and returns it to the calling program - as if it were a normal return value.

Example: In gRPC, you define a service in a .proto file (`rpc GetBalance (AccountRequest) returns (BalanceResponse);`). The compiler generates client and server stubs automatically. Your client code just calls `client.GetBalance(request)` - gRPC handles serialization (via Protocol Buffers), network transmission, and deserialization behind the scenes.

Why it matters: it gives *transparency* - the programmer writes distributed code almost like local code.

3. Difference between Client and Server

Aspect	Client	Server
Role	Requests a service/resource	Provides the service/resource
Initiates communication	Yes - sends the first request	No - listens for requests
Number of instances	Usually many	Usually one (or a small cluster)
State	Often stateless between requests, or holds session state locally	Typically manages shared state/data (e.g., a database)
Resource ownership	Doesn't own the shared resource	Owns and controls access to the resource
Example	Web browser	Web server (e.g., Nginx)

Example: When you open gmail.com in your browser, your browser (client) sends an HTTP request to Google's mail servers. The server processes it, fetches your emails from its database, and sends back an HTML response. The browser never directly touches Google's storage - it must go through the server.

4. What is the Name Space

A **name space** is a set of names (identifiers) together with a mapping (or set of rules) that resolves those names to the actual entities/objects they refer to - files, processes, services, machines, etc. In a distributed system, name spaces let different nodes refer to shared resources consistently, even though those resources may move or be replicated.

Key ideas:

- Names can be **flat** (unstructured, like a random UID) or **structured/hierarchical** (like file paths or domain names).
- Name resolution** is the process of looking up what a name actually refers to (e.g., an IP address, a memory location, a file handle).
- Name spaces can be **distributed** themselves (no single machine holds the whole mapping) for scalability and fault tolerance.

Example: The Domain Name System (DNS) is a classic distributed name space. www.google.com is a hierarchical name. When you type it into a browser, DNS resolves it to an IP address (e.g., 142.250.190.4) through a hierarchy of name servers (root -> .com -> google.com) - none of which holds the entire mapping alone.

5. Explain Election Algorithm

In distributed systems, many algorithms need one process to act as a **coordinator/leader** (e.g., to manage mutual exclusion, or replace a failed coordinator). An **election algorithm** is used to pick that coordinator among a group of processes, especially important when the previous coordinator has crashed.

Requirement: All processes must agree on the same coordinator, even though they can't see each other's internal state directly.

a) Bully Algorithm

Each process has a unique priority number (e.g., process ID). The process with the highest ID becomes coordinator.

- When a process notices the coordinator is down, it sends an "Election" message to all processes with a higher ID.
- If no one with a higher ID responds, it declares itself the coordinator and announces this to everyone.
- If a higher-ID process responds, that process takes over the election and repeats the process.
- The process with the highest ID among the live ones wins and broadcasts "I am coordinator."

Example: Processes P1, P2, P3, P4, P5 (P5 = highest ID = current coordinator). If P5 crashes, P3 notices timeouts and sends "Election" to P4 and P5. P5 doesn't respond (it's down), P4 responds and takes over, becomes new coordinator (since no one higher responds), and announces itself.

b) Ring Algorithm

Processes are logically arranged in a ring. Each process knows only its successor.

- A process that detects the coordinator is down starts an election by sending its own ID around the ring.
- Each process compares the received ID with its own - it forwards whichever is larger, appending its own ID to a list if useful.
- When the message returns to the process that started it, the highest ID seen is elected coordinator.
- This is announced around the ring.

Example: In a 5-node ring (P1->P2->P3->P4->P5->P1), if P1 starts an election, the ID travels around the ring; each node compares/forwards the larger ID. By the time it circles back to P1, the largest ID has been identified as coordinator, and a second message confirms this to all.

6. Overview of Distributed Operating Systems

A **Distributed Operating System** (DOS) manages a collection of independent, networked computers and presents them to users and applications as if they were a single coherent computer system. It builds on the concept of distributed systems but goes further by unifying resource management, scheduling, and file access across machines.

Key goals/features:

- **Resource sharing** - CPU, memory, storage, and devices across machines are managed collectively.
- **Transparency** - users shouldn't notice that resources are spread across multiple machines: access transparency, location transparency, migration transparency, replication transparency.
- **Concurrency** - multiple processes on different nodes can run and interact simultaneously.
- **Fault tolerance** - the system continues to function even if individual nodes fail.
- **Scalability** - should work efficiently as more nodes are added.
- **Openness** - supports interoperability using standard protocols.

Components typically found in a DOS:

- **Distributed file system** (e.g., NFS, Google File System) - files accessible from any node
- **Distributed process management** - process creation/scheduling across nodes

- **Distributed shared memory / message passing** - coordination mechanisms
- **Naming & directory services** - to locate resources
- **Security & synchronization mechanisms** - distributed mutual exclusion, clock synchronization

Example: Google's internal infrastructure (using systems like Borg/Kubernetes-like schedulers, GFS, Chubby for locking) behaves like one giant OS spanning thousands of machines - you submit a job, and the "OS" decides which physical machine runs it, handles failures, and manages storage transparently. Another classic teaching example is Amoeba or Sprite, academic distributed OSes designed explicitly around these transparency principles.